شرح سطر بسطر ٢ — قاعدة البيانات المحلية (Drift)

مشروع رِزمة · Flutter · كل سطر مع سبب كتابته بهذه الطريقة

كل ما يخص الطلاب (ملفاتهم وتقدّمهم ومحاولاتهم) يُحفظ على الجهاز في SQLite عبر مكتبة Drift. هذا الملف يشرح الجداول الستة والتعدادات وصنف القاعدة والمزوّدات والاستعلامات (DAOs).

مبدأ مهم: كل جدول بيانات يحمل `profileId` وكل استعلام مفلتر به، فلا يرى طالب تقدّم أخيه على الهاتف المشترك. وعند حذف ملف تُحذف بياناته تلقائيًا.

ملفات `*.g.dart` تُولَّد تلقائيًا بالأمر `dart run build_runner build` ولا تُكتب يدويًا، لذلك لا نشرحها.

lib/core/db/enums.dart — التعدادات

lib/core/db/enums.dart

الأسطر 1–2

```
1  /// ألوان الشخصية. تُخزَّن بالاسم النصي في قاعدة البيانات.
2  enum ProfileColor { teal, coral, blue, purple, amber }
```

ماذا يفعل؟ تعداد `ProfileColor` بخمسة ألوان للشخصية.

لماذا بهذه الطريقة؟ يُخزَّن في القاعدة بالاسم النصي (`textEnum`) فلا يتأثر لو أعدنا ترتيب القيم لاحقًا.

السطر 4

```
4  enum LessonStatus { notStarted, downloaded, inProgress, completed }
```

ماذا يفعل؟ تعداد `LessonStatus`: لم يبدأ / محمَّل / قيد التقدّم / مكتمل.

لماذا بهذه الطريقة؟ أربع حالات تكفي لوصف رحلة الدرس. «محمَّل» لأن التطبيق يعمل دون إنترنت وينزّل الدروس مسبقًا.

السطر 6

```
6  enum AttemptKind { practice, quiz }
```

ماذا يفعل؟ تعداد `AttemptKind`: تدريب أو اختبار.

لماذا بهذه الطريقة؟ التمييز مهم: قسم المراجعة لا يظهر إلا بعد اختبار.

lib/core/db/tables/profiles.dart — جدول ملفات الطلاب

lib/core/db/tables/profiles.dart

الأسطر 1–3

```dart
import 'package:drift/drift.dart';

import '../enums.dart';
```

ماذا يفعل؟ استيراد Drift والتعدادات.

لماذا بهذه الطريقة؟ نحتاج `Table` وأنواع الأعمدة.

الأسطر 5–6

```dart
/// ملف طالب على هذا الجهاز. لا بريد ولا كلمة مرور ولا رقم هاتف.
class Profiles extends Table {
```

ماذا يفعل؟ تعليق وصنف `Profiles` يرث `Table`.

لماذا بهذه الطريقة؟ اسم الصنف يصير اسم الجدول، وتولّد Drift فئة البيانات `Profile`. لا بريد ولا كلمة مرور ولا هاتف: الملف هو «الحساب».

السطر 7

```dart
  TextColumn get id => text()();
```

ماذا يفعل؟ العمود `id` نص.

لماذا بهذه الطريقة؟ معرّف uuid (مفتاح أساسي). لا نستعمل الاسم لأنه قد يتكرر أو يتغيّر.

السطر 8

```dart
  TextColumn get name => text().withLength(min: 1, max: 40)();
```

ماذا يفعل؟ العمود `name` بطول بين 1 و40.

لماذا بهذه الطريقة؟ قيد يمنع الأسماء الفارغة أو الطويلة جدًا حتى من داخل القاعدة.

الأسطر 9–10

```dart
  // ignore: recursive_getters
  IntColumn get grade => integer().check(grade.isBetweenValues(1, 12))();
```

ماذا يفعل؟ العمود `grade` عدد صحيح مع `CHECK` بين 1 و12. وسطر `ignore` لتجاهل تنبيه المحلل.

لماذا بهذه الطريقة؟ التحقق داخل القاعدة يحمي حتى لو أخطأ كود آخر. التنبيه `recursive_getters` وهمي: Drift تفسّر `grade` داخل `check` كعمود لا كاستدعاء ذاتي.

السطر 11

```dart
  TextColumn get color => textEnum<ProfileColor>()();
```

ماذا يفعل؟ العمود `color` تعداد نصي.

لماذا بهذه الطريقة؟ `textEnum` يخزّن الاسم («teal») ويعيده كـ `ProfileColor`.

الأسطر 13–15

```
13    /// SHA-256 للرمز السري مع salt عشوائي. null = لا رمز سري.
14    TextColumn get pinHash => text().nullable()();
15    TextColumn get pinSalt => text().nullable()();
```

ماذا يفعل؟ عمودا `pinHash` و`pinSalt` يقبلان `null`.

لماذا بهذه الطريقة؟ الرمز السري **مجزّأ** مع salt عشوائي؛ `null` يعني لا رمز سري. لا يُخزَّن الرمز نصًّا أبدًا.

السطر 17

```
17    TextColumn get studentCode => text().unique()();
```

ماذا يفعل؟ العمود `studentCode` فريد (`unique`).

لماذا بهذه الطريقة؟ رمز الطالب `RZ-XXXX` يجب ألا يتكرر على الجهاز.

السطر 18

```
18    TextColumn get classCode => text().nullable()();
```

ماذا يفعل؟ `classCode` يقبل `null`.

لماذا بهذه الطريقة؟ رمز الصف من المعلم اختياري.

الأسطر 19–20

```
19    DateTimeColumn get createdAt => dateTime()();
20    DateTimeColumn get lastActiveAt => dateTime()();
```

ماذا يفعل؟ تاريخا الإنشاء وآخر نشاط.

لماذا بهذه الطريقة؟ `lastActiveAt` يرتّب شاشة الاختيار (الأحدث أولًا) ويحدّد الملف الأخير.

الأسطر 22–24

```
22    @override
23    Set<Column> get primaryKey => {id};
24  }
```

ماذا يفعل؟ المفتاح الأساسي هو `id` ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ نحدّده صراحةً لأن العمود نص وليس `autoIncrement`.

lib/core/db/tables/lesson_progress.dart — تقدّم الدروس

lib/core/db/tables/lesson_progress.dart

الأسطر 1–4

```
import 'package:drift/drift.dart';

import '../enums.dart';
import 'profiles.dart';
```

ماذا يفعل؟ استيراد Drift والتعدادات وجدول الملفات.

لماذا بهذه الطريقة؟ الجدول يشير لـ `Profiles` بمفتاح أجنبي.

الأسطر 6–8

```
/// تقدّم كل طالب في كل درس. تُحذف صفوف الطالب مع حذف ملفه.
@DataClassName('LessonProgressEntry')
class LessonProgress extends Table {
```

ماذا يفعل؟ تعليق و `DataClassName` وصنف الجدول.

لماذا بهذه الطريقة؟ نحدّد اسم فئة البيانات صراحةً `LessonProgressEntry` بدل الاسم الذي تشتقه Drift تلقائيًا من اسم الجدول، لأن الاسم المفرد المشتق لكلمة تنتهي بـ Progress غير واضح.

الأسطر 9–10

```
  TextColumn get profileId =>
      text().references(Profiles, #id, onDelete: KeyAction.cascade)();
```

ماذا يفعل؟ `profileId` مفتاح أجنبي إلى `Profiles.id` مع الحذف المتسلسل.

لماذا بهذه الطريقة؟ عند حذف الملف تُحذف صفوف تقدّمه، فلا تبقى بيانات يتيمة.

السطر 11

```
  TextColumn get lessonId => text()();
```

ماذا يفعل؟ `lessonId` نص.

لماذا بهذه الطريقة؟ معرّف الدرس من الكتالوج (مثل `math.u1.l1`).

السطر 12

```
  TextColumn get status => textEnum<LessonStatus>()();
```

ماذا يفعل؟ `status` تعداد.

لماذا بهذه الطريقة؟ حالة الدرس.

السطر 13

```
  RealColumn get progress => real().withDefault(const Constant(0))();
```

ماذا يفعل؟ `progress` رقم عشري افتراضيه 0.

لماذا بهذه الطريقة؟ نسبة التقدّم 0..1.

السطر 14

```
14    DateTimeColumn get updatedAt => dateTime()();
```

ماذا يفعل؟ `updatedAt` تاريخ.

لماذا بهذه الطريقة؟ يحدّد «آخر درس» في بطاقة المتابعة ويدخل في حساب السلسلة.

الأسطر 16–18

```
16    @override
17    Set<Column> get primaryKey => {profileId, lessonId};
18  }
```

ماذا يفعل؟ مفتاح أساسي مركّب `(profileId, lessonId)` .

لماذا بهذه الطريقة؟ صف واحد لكل طالب لكل درس؛ يسمح بالكتابة بـ `insertOnConflictUpdate` (إنشاء أو تحديث).

lib/core/db/tables/attempts.dart — المحاولات

lib/core/db/tables/attempts.dart

الأسطر 1–4

```
1   import 'package:drift/drift.dart';
2
3   import '../enums.dart';
4   import 'profiles.dart';
```

ماذا يفعل؟ استيراد Drift والتعدادات وجدول الملفات.

لماذا بهذه الطريقة؟ نفس سبب الجدول السابق.

الأسطر 6–7

```
6   /// مجموع النجوم. محاولات التدريب والاختبار — `starsEarned` (تُحسب ولا تُخزَّن).
7   class Attempts extends Table {
```

ماذا يفعل؟ تعليق وصنف `Attempts` .

لماذا بهذه الطريقة؟ النجوم **تُحسب** (مجموع `starsEarned`) ولا تُخزَّن كرقم مستقل.

السطر 8

```
8     IntColumn get id => integer().autoIncrement()();
```

ماذا يفعل؟ `id` عدد يتزايد تلقائيًا.

لماذا بهذه الطريقة؟ المحاولات كثيرة ولا حاجة لمعرّف نصي طويل.

الأسطر 9–10

```
9       TextColumn get profileId =>
10          text().references(Profiles, #id, onDelete: KeyAction.cascade)();
```

ماذا يفعل؟ `profileId` مفتاح أجنبي مع حذف متسلسل.

لماذا بهذه الطريقة؟ محاولات كل طالب مرتبطة بملفه.

السطر 11

```
11      TextColumn get activityId => text()();
```

ماذا يفعل؟ `activityId` نص.

لماذا بهذه الطريقة؟ معرّف النشاط (تدريب أو اختبار) لاحقًا.

السطر 12

```
12      TextColumn get kind => textEnum<AttemptKind>()();
```

ماذا يفعل؟ `kind` تعداد: تدريب أو اختبار.

لماذا بهذه الطريقة؟ الاختبار هو الذي يفتح قسم المراجعة.

الأسطر 13–14

```
13      IntColumn get score => integer()();
14      IntColumn get total => integer()();
```

ماذا يفعل؟ `score` و `total`.

لماذا بهذه الطريقة؟ عدد الإجابات الصحيحة من الإجمالي.

السطر 15

```
15      IntColumn get starsEarned => integer().withDefault(const Constant(0))();
```

ماذا يفعل؟ `starsEarned` افتراضيه 0.

لماذا بهذه الطريقة؟ نجوم هذه المحاولة؛ مجموعها هو رصيد الطالب.

الأسطر 16–17

```
16      DateTimeColumn get createdAt => dateTime()();
17  }
```

ماذا يفعل؟ `createdAt` وإغلاق الصنف.

لماذا بهذه الطريقة؟ يدخل في حساب السلسلة.

lib/core/db/tables/concept_mastery.dart — إتقان المفاهيم

`lib/core/db/tables/concept_mastery.dart`

الأسطر 1–3

```
import 'package:drift/drift.dart';

import 'profiles.dart';
```

ماذا يفعل؟ استيراد Drift وجدول الملفات.

لماذا بهذه الطريقة؟ لا حاجة للتعدادات هنا.

الأسطر 5–7

```
/// درجة إتقان الطالب لكل مفهوم (0..1). منها يُبنى قسم «يحتاج إلى مراجعة».
@DataClassName('ConceptMasteryEntry')
class ConceptMastery extends Table {
```

ماذا يفعل؟ تعليق و `DataClassName` وصنف الجدول.

لماذا بهذه الطريقة؟ نحدّد اسم فئة البيانات `ConceptMasteryEntry` ليكون واضحًا ومتّسقًا مع بقية الجداول.

الأسطر 8–9

```
  TextColumn get profileId =>
      text().references(Profiles, #id, onDelete: KeyAction.cascade)();
```

ماذا يفعل؟ `profileId` مفتاح أجنبي مع حذف متسلسل.

لماذا بهذه الطريقة؟ إتقان كل طالب مستقل.

السطر 10

```
  TextColumn get conceptId => text()();
```

ماذا يفعل؟ `conceptId` نص.

لماذا بهذه الطريقة؟ معرّف المفهوم من الكتالوج.

السطر 11

```
  RealColumn get mastery => real()();
```

ماذا يفعل؟ `mastery` رقم عشري 0..1.

لماذا بهذه الطريقة؟ منه يُبنى قسم «يحتاج إلى مراجعة» (أقل من 0.6).

السطر 12

```
  DateTimeColumn get updatedAt => dateTime()();
```

ماذا يفعل؟ `updatedAt` .

لماذا بهذه الطريقة؟ وقت آخر تحديث.

الأسطر 14–16

```
  @override
  Set<Column> get primaryKey => {profileId, conceptId};
}
```

ماذا يفعل؟ مفتاح أساسي مركّب `(profileId, conceptId)`.

لماذا بهذه الطريقة؟ قيمة إتقان واحدة لكل طالب لكل مفهوم.

lib/core/db/tables/sync_queue.dart — طابور المزامنة

lib/core/db/tables/sync_queue.dart

الأسطر 1–3

```
import 'package:drift/drift.dart';

/// طابور المزامنة. يمتلئ الآن ولا يُرسل؛ الإرسال في مرحلة لاحقة.
```

ماذا يفعل؟ استيراد Drift وتعليق الغرض.

لماذا بهذه الطريقة؟ يمتلئ الآن ولا يُرسل؛ الإرسال لاحقًا (Supabase).

الأسطر 4–5

```
@DataClassName('SyncQueueEntry')
class SyncQueue extends Table {
```

ماذا يفعل؟ `DataClassName` وصنف `SyncQueue`.

لماذا بهذه الطريقة؟ الاسم `SyncQueueEntry`.

السطر 6

```
  IntColumn get id => integer().autoIncrement()();
```

ماذا يفعل؟ `id` يتزايد تلقائيًا.

لماذا بهذه الطريقة؟ ترتيب الدخول = ترتيب الإرسال.

الأسطر 7–8

```
  TextColumn get entity => text()();
  TextColumn get payloadJson => text()();
```

ماذا يفعل؟ `entity` و`payloadJson`.

لماذا بهذه الطريقة؟ نوع العنصر (مثل محاولة) وبياناته كـ JSON، فلا نربط الطابور بجداول معيّنة.

السطر 9

```
  DateTimeColumn get createdAt => dateTime()();
```

ماذا يفعل؟ `createdAt`.

لماذا بهذه الطريقة؟ لمعرفة عمر العنصر.

الأسطر 10–11

```
  IntColumn get tries => integer().withDefault(const Constant(0))();
}
```

ماذا يفعل؟ `tries` افتراضيه 0 وإغلاق الصنف.

لماذا بهذه الطريقة؟ عدد محاولات الإرسال الفاشلة لاحقًا.

lib/core/db/tables/app_settings.dart — إعدادات الجهاز

lib/core/db/tables/app_settings.dart

الأسطر 1–3

```
import 'package:drift/drift.dart';

/// إعدادات عامة للجهاز على شكل مفتاح/قيمة (مثل الملف النشط الأخير).
```

ماذا يفعل؟ استيراد Drift وتعليق الغرض.

لماذا بهذه الطريقة؟ إعدادات تخص الجهاز لا الطالب (مثل الملف النشط الأخير).

الأسطر 4–5

```
@DataClassName('AppSetting')
class AppSettings extends Table {
```

ماذا يفعل؟ `DataClassName` وصنف `AppSettings`.

لماذا بهذه الطريقة؟ فئة البيانات `AppSetting`.

الأسطر 6–7

```
  TextColumn get key => text()();
  TextColumn get value => text()();
```

ماذا يفعل؟ عمودان نصيان: المفتاح والقيمة.

لماذا بهذه الطريقة؟ مفتاح/قيمة بسيط يتسع لأي إعداد جديد دون تعديل المخطط.

الأسطر 9–11

```
  @override
  Set<Column> get primaryKey => {key};
}
```

ماذا يفعل؟ المفتاح الأساسي هو `key` وإغلاق الصنف.

لماذا بهذه الطريقة؟ قيمة واحدة لكل مفتاح، فنكتب بـ `insertOnConflictUpdate`.

lib/core/db/app_database.dart — صنف القاعدة

lib/core/db/app_database.dart

الأسطر 1–2

```
1   import 'package:drift/drift.dart';
2   import 'package:drift_flutter/drift_flutter.dart';
```

ماذا يفعل؟ استيراد Drift و `drift_flutter`.

لماذا بهذه الطريقة؟ `drift_flutter` تفتح ملف SQLite المناسب لكل منصة.

الأسطر 4–13

```
4   import 'daos/activity_dao.dart';
5   import 'daos/profiles_dao.dart';
6   import 'daos/settings_dao.dart';
7   import 'enums.dart';
8   import 'tables/app_settings.dart';
9   import 'tables/attempts.dart';
10  import 'tables/concept_mastery.dart';
11  import 'tables/lesson_progress.dart';
12  import 'tables/profiles.dart';
13  import 'tables/sync_queue.dart';
```

ماذا يفعل؟ استيراد الـ DAOs والتعدادات والجداول الستة.

لماذا بهذه الطريقة؟ ما يعلنه `@DriftDatabase` يجب أن يكون مستوردًا هنا.

السطر 15

```
15  export 'enums.dart';
```

ماذا يفعل؟ إعادة تصدير `enums.dart`.

لماذا بهذه الطريقة؟ من يستورد `app_database.dart` يحصل على التعدادات تلقائيًا فلا يحتاج استيراد ثانٍ.

السطر 17

```
17  part 'app_database.g.dart';
```

ماذا يفعل؟ جزء الكود المولَّد.

لماذا بهذه الطريقة؟ ملف `app_database.g.dart` من Drift فيه `_$AppDatabase`.

الأسطر 19–29

```
@DriftDatabase(
  tables: [
    Profiles,
    LessonProgress,
    Attempts,
    ConceptMastery,
    SyncQueue,
    AppSettings,
  ],
  daos: [ProfilesDao, ActivityDao, SettingsDao],
)
```

ماذا يفعل؟ التعليق التوضيحي `@DriftDatabase` يسرد الجداول الستة والـ DAOs الثلاثة.

لماذا بهذه الطريقة؟ من هذه القائمة تُولِّد الاستعلامات وفئات البيانات.

الأسطر 30–31

```
class AppDatabase extends _$AppDatabase {
  AppDatabase(super.executor);
```

ماذا يفعل؟ صنف `AppDatabase` وباني يقبل «منفّذ» `executor`.

لماذا بهذه الطريقة؟ قبول المنفّذ يسمح بتمرير قاعدة في الذاكرة في الاختبارات.

الأسطر 33–44

```
  /// قاعدة البيانات الحقيقية على الجهاز (SQLite).
  AppDatabase.onDevice()
    : super(
        driftDatabase(
          name: 'rizma',
          // للتطوير على الويب فقط (Chrome): يحتاج الملفين في مجلد web/.
          web: DriftWebOptions(
            sqlite3Wasm: Uri.parse('sqlite3.wasm'),
            driftWorker: Uri.parse('drift_worker.js'),
          ),
        ),
      );
```

ماذا يفعل؟ باني مسمّى `onDevice` يفتح القاعدة الحقيقية باسم `rizma`، مع إعداد الويب (`sqlite3.wasm` و`drift_worker.js`).

لماذا بهذه الطريقة؟ على الجوال يُتجاهل `web`. على Chrome لا تعمل Drift بدون هذين الملفين في مجلد `web/` (وإلا فشل إنشاء الملف).

الأسطر 46–47

```
  @override
  int get schemaVersion => 1;
```

ماذا يفعل؟ `schemaVersion = 1`.

لماذا بهذه الطريقة؟ رقم نسخة المخطط. عند تعديل أي جدول نرفعه ونضيف ترحيلًا (`onUpgrade`) قبل أي إصدار يُوزَّع.

الأسطر 49–55

```
49    @override
50    MigrationStrategy get migration => MigrationStrategy(
51      beforeOpen: (details) async {
52        // بدونها لا يعمل الحذف المتسلسل عند حذف ملف طالب.
53        await customStatement('PRAGMA foreign_keys = ON');
54      },
55    );
```

ماذا يفعل؟ `beforeOpen` ينفّذ `PRAGMA foreign_keys = ON` .

لماذا بهذه الطريقة؟ SQLite يعطّل المفاتيح الأجنبية افتراضيًا؛ بدونها لا يعمل الحذف المتسلسل (`onDelete: cascade`).

السطر 56

```
56  }
```

ماذا يفعل؟ إغلاق الصنف.

لماذا بهذه الطريقة؟ نهاية الملف.

lib/core/db/database_provider.dart — مزوّدا القاعدة والساعة

lib/core/db/database_provider.dart

الأسطر 1–3

```
1   import 'package:flutter_riverpod/flutter_riverpod.dart';
2
3   import 'app_database.dart';
```

ماذا يفعل؟ استيراد Riverpod وصنف القاعدة.

لماذا بهذه الطريقة؟ المزوّد يوزّع القاعدة على كل التطبيق.

الأسطر 5–10

```
5   /// قاعدة البيانات الوحيدة للتطبيق. تُستبدل في الاختبارات بقاعدة في الذاكرة.
6   final appDatabaseProvider = Provider<AppDatabase>((ref) {
7     final db = AppDatabase.onDevice();
8     ref.onDispose(db.close);
9     return db;
10  });
```

ماذا يفعل؟ `appDatabaseProvider` ينشئ `AppDatabase.onDevice()` ويغلقها عند التخلّص.

لماذا بهذه الطريقة؟ نسخة واحدة للتطبيق. `ref.onDispose(db.close)` يضمن إغلاقها. في الاختبارات نستبدل هذا المزوّد بقاعدة في الذاكرة.

الأسطر 12–13

```
/// الساعة الحالية، قابلة للاستبدال في الاختبارات.
final clockProvider = Provider<DateTime Function()>((ref) => DateTime.now);
```

ماذا يفعل؟ `clockProvider` يعيد الدالة `DateTime.now`.

لماذا بهذه الطريقة؟ الساعة مزوّدة قابلة للاستبدال، فنختبر السلسلة والقفل بزمن مضبوط دون انتظار حقيقي.

lib/core/db/daos/profiles_dao.dart — استعلامات الملفات

lib/core/db/daos/profiles_dao.dart

الـ DAO يقرأ ويكتب فقط. قواعد العمل (التحقق، القفل...) في المستودع `ProfileRepository`.

الأسطر 1–6

```
import 'package:drift/drift.dart';

import '../app_database.dart';
import '../tables/profiles.dart';

part 'profiles_dao.g.dart';
```

ماذا يفعل؟ استيراد Drift والقاعدة والجدول وجزء الكود المولَّد.

لماذا بهذه الطريقة؟ تُولَّد `_$ProfilesDaoMixin` في الملف المولَّد.

الأسطر 8–11

```
@DriftAccessor(tables: [Profiles])
class ProfilesDao extends DatabaseAccessor<AppDatabase>
    with _$ProfilesDaoMixin {
  ProfilesDao(super.db);
```

ماذا يفعل؟ `@DriftAccessor` على الجدول وصنف `ProfilesDao` وباني يمرّر القاعدة.

لماذا بهذه الطريقة؟ يوفّر دوال الاستعلام للجدول `profiles` مع وصول إلى القاعدة.

الأسطر 13–17

```
  Stream<List<Profile>> watchAll() {
    return (select(
      profiles,
    )..orderBy([(p) => OrderingTerm.desc(p.lastActiveAt)])).watch();
  }
```

ماذا يفعل؟ `watchAll`: بث (`Stream`) بكل الملفات مرتبةً بآخر نشاط تنازليًا.

لماذا بهذه الطريقة؟ البث يعيد إصدار القائمة عند أي تغيير فتتحدّث الشاشة وحدها.

الأسطر 19–23

```
19    Future<List<Profile>> getAll() {
20      return (select(
21        profiles,
22      )..orderBy([(p) => OrderingTerm.desc(p.lastActiveAt)])).get();
23    }
```

ماذا يفعل؟ `getAll` : نفس الاستعلام لكن قراءة واحدة.

لماذا بهذه الطريقة؟ يُستعمل عند الإقلاع لاتخاذ القرار.

الأسطر 25–27

```
25    Future<Profile?> getById(String id) {
26      return (select(profiles)..where((p) => p.id.equals(id))).getSingleOrNull();
27    }
```

ماذا يفعل؟ `getById` : ملف بمعرّفه أو `null` .

لماذا بهذه الطريقة؟ `getSingleOrNull` لا يرمي خطأً إن لم يوجد.

الأسطر 29–33

```
29    Stream<Profile?> watchById(String id) {
30      return (select(
31        profiles,
32      )..where((p) => p.id.equals(id))).watchSingleOrNull();
33    }
```

ماذا يفعل؟ `watchById` : بث لملف واحد.

لماذا بهذه الطريقة؟ الملف النشط يتحدّث تلقائيًا عند تعديل بياناته.

الأسطر 35–39

```
35    Future<Profile?> getByStudentCode(String code) {
36      return (select(
37        profiles,
38      )..where((p) => p.studentCode.equals(code))).getSingleOrNull();
39    }
```

ماذا يفعل؟ `getByStudentCode` : بحث برمز الطالب.

لماذا بهذه الطريقة؟ للتحقق من التفرّد ومن «نسيت الرمز».

الأسطر 41–47

```
41    Future<int> count() async {
42      final countExp = profiles.id.count();
43      final row = await (selectOnly(
44        profiles,
45      )..addColumns([countExp])).getSingle();
46      return row.read(countExp) ?? 0;
47    }
```

ماذا يفعل؟ `count` : عدد الملفات بـ `selectOnly` و `count()` .

لماذا بهذه الطريقة؟ يحسب في القاعدة دون جلب كل الصفوف.

الأسطر 49–51

```
49    Future<void> insertProfile(ProfilesCompanion entry) async {
50      await into(profiles).insert(entry);
51    }
```

ماذا يفعل؟ `insertProfile` : إضافة ملف.

لماذا بهذه الطريقة؟ تستقبل `ProfilesCompanion` (قيم قابلة لأن تكون غير محدّدة).

الأسطر 53–61

```
53    Future<void> setPin(
54      String id, {
55      required String hash,
56      required String salt,
57    }) async {
58      await (update(profiles)..where((p) => p.id.equals(id))).write(
59        ProfilesCompanion(pinHash: Value(hash), pinSalt: Value(salt)),
60      );
61    }
```

ماذا يفعل؟ `setPin` : تعيين التجزئة والـ salt لملف.

لماذا بهذه الطريقة؟ `Value(...)` تعني «حدّث هذا العمود»؛ الأعمدة غير المذكورة لا تتغيّر.

الأسطر 63–67

```
63    Future<void> clearPin(String id) async {
64      await (update(profiles)..where((p) => p.id.equals(id))).write(
65        const ProfilesCompanion(pinHash: Value(null), pinSalt: Value(null)),
66      );
67    }
```

ماذا يفعل؟ `clearPin` : يجعل التجزئة والـ `null` salt.

لماذا بهذه الطريقة؟ `Value(null)` تعني «اكتب NULL» وتختلف عن `Value.absent()` (لا تغيّر).

الأسطر 69–73

```
69    Future<void> setGrade(String id, int grade) async {
70      await (update(profiles)..where((p) => p.id.equals(id))).write(
71        ProfilesCompanion(grade: Value(grade)),
72      );
73    }
```

ماذا يفعل؟ `setGrade`: يحدّث عمود `grade` لملف.

لماذا بهذه الطريقة؟ `write` بـ `ProfilesCompanion` فيها `grade` فقط: بقية الأعمدة (الاسم والرمز السري والنجوم عبر المحاولات) لا تُلمس. التحقق من النطاق 1–12 في المستودع، وقيد `check` في الجدول حارس أخير.

الأسطر 75–77

```
75    Future<void> deleteById(String id) async {
76      await (delete(profiles)..where((p) => p.id.equals(id))).go();
77    }
```

ماذا يفعل؟ `deleteById`: يحذف صفّ ملف بمعرّفه.

لماذا بهذه الطريقة؟ `delete(...)..where(...).go()`. الجداول المرتبطة (التقدّم والمحاولات والإتقان) فيها `onDelete: cascade` فتُحذف معه تلقائيًا، لكن المستودع يمسحها صراحةً أيضًا في المعاملة نفسها (حزام وحمّالة).

الأسطر 79–83

```
79    Future<void> touch(String id, DateTime when) async {
80      await (update(profiles)..where((p) => p.id.equals(id))).write(
81        ProfilesCompanion(lastActiveAt: Value(when)),
82      );
83    }
```

ماذا يفعل؟ `touch`: تحديث `lastActiveAt`.

لماذا بهذه الطريقة؟ يُستدعى عند اختيار الملف ليصير الأحدث.

الأسطر 85–90

```
85    Future<void> setClassCode(String id, String? classCode) async {
86      await (update(profiles)..where((p) => p.id.equals(id))).write(
87        ProfilesCompanion(classCode: Value(classCode)),
88      );
89    }
90  }
```

ماذا يفعل؟ `setClassCode` ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ يقبل `null` لإلغاء رمز الصف.

lib/core/db/daos/activity_dao.dart — استعلامات النشاط

lib/core/db/daos/activity_dao.dart

الأسطر 1–9

```dart
import 'package:drift/drift.dart';

import '../app_database.dart';
import '../tables/attempts.dart';
import '../tables/concept_mastery.dart';
import '../tables/lesson_progress.dart';
import '../tables/sync_queue.dart';

part 'activity_dao.g.dart';
```

ماذا يفعل؟ استيرادات Drift والقاعدة والجداول الأربعة وجزء الكود المولَّد.

لماذا بهذه الطريقة؟ لا نستورد التعدادات لأن `app_database.dart` تعيد تصديرها.

الأسطر 11–13

```dart
/// كل ما يخص نشاط الطالب: تقدّم الدروس، المحاولات، الإتقان، وطابور المزامنة.
/// كل الاستعلامات مفلترة بـ `profileId` فلا يرى طالب بيانات أخيه.
@DriftAccessor(tables: [LessonProgress, Attempts, ConceptMastery, SyncQueue])
```

ماذا يفعل؟ تعليق و `@DriftAccessor` للجداول الأربعة.

لماذا بهذه الطريقة؟ كل الاستعلامات هنا مفلترة بـ `profileId` فلا يرى طالب بيانات أخيه.

الأسطر 14–16

```dart
class ActivityDao extends DatabaseAccessor<AppDatabase>
    with _$ActivityDaoMixin {
  ActivityDao(super.db);
```

ماذا يفعل؟ صنف `ActivityDao` وبانيه.

لماذا بهذه الطريقة؟ نفس نمط DAO السابق.

الأسطر 18–23

```dart
  // تقدّم الدروس
  Stream<List<LessonProgressEntry>> watchLessonProgress(String profileId) {
    return (select(
      lessonProgress,
    )..where((r) => r.profileId.equals(profileId))).watch();
  }
```

ماذا يفعل؟ `watchLessonProgress`: بث بتقدّم الدروس لملف.

لماذا بهذه الطريقة؟ الشرط `profileId.equals(...)` هو ضمان الفصل بين الطلاب.

الأسطر 25–41

```
  Future<void> upsertLessonProgress({
    required String profileId,
    required String lessonId,
    required LessonStatus status,
    required double progress,
    required DateTime now,
  }) async {
    await into(lessonProgress).insertOnConflictUpdate(
      LessonProgressCompanion.insert(
        profileId: profileId,
        lessonId: lessonId,
        status: status,
        progress: Value(progress),
        updatedAt: now,
      ),
    );
  }
```

ماذا يفعل؟ `upsertLessonProgress` : كتابة تقدّم درس (إنشاء أو تحديث).

لماذا بهذه الطريقة؟ `insertOnConflictUpdate` يحدّث الصف إن وُجد المفتاح المركّب. `Value(progress)` لأن العمود له قيمة افتراضية.

الأسطر 43–48

```
  // المحاولات
  Stream<List<Attempt>> watchAttempts(String profileId) {
    return (select(
      attempts,
    )..where((a) => a.profileId.equals(profileId))).watch();
  }
```

ماذا يفعل؟ `watchAttempts` : بث بمحاولات ملف.

لماذا بهذه الطريقة؟ مصدر النجوم والسلسلة وخطوات «الأولى».

الأسطر 50–70

```
  Future<int> insertAttempt({
    required String profileId,
    required String activityId,
    required AttemptKind kind,
    required int score,
    required int total,
    required int starsEarned,
    required DateTime now,
  }) {
    return into(attempts).insert(
      AttemptsCompanion.insert(
        profileId: profileId,
        activityId: activityId,
        kind: kind,
        score: score,
        total: total,
        starsEarned: Value(starsEarned),
        createdAt: now,
      ),
    );
  }
```

ماذا يفعل؟ `insertAttempt` : إضافة محاولة وإرجاع معرّفها.

لماذا بهذه الطريقة؟ تعيد `Future<int>` (المعرّف الجديد).

الأسطر 72–77

```
  // الإتقان
  Stream<List<ConceptMasteryEntry>> watchMastery(String profileId) {
    return (select(
      conceptMastery,
    )..where((r) => r.profileId.equals(profileId))).watch();
  }
```

ماذا يفعل؟ `watchMastery` : بث بإتقان المفاهيم لملف.

لماذا بهذه الطريقة؟ منه تُبنى المراجعة.

الأسطر 79–93

```
  Future<void> upsertMastery({
    required String profileId,
    required String conceptId,
    required double mastery,
    required DateTime now,
  }) async {
    await into(conceptMastery).insertOnConflictUpdate(
      ConceptMasteryCompanion.insert(
        profileId: profileId,
        conceptId: conceptId,
        mastery: mastery,
        updatedAt: now,
      ),
    );
  }
```

ماذا يفعل؟ `upsertMastery` : كتابة إتقان مفهوم.

لماذا بهذه الطريقة؟ نفس نمط تقدّم الدروس.

الأسطر 95–108

```
  /// مجموع النجوم لكل ملف (يظهر في شاشة «من يتعلّم الآن؟»).
  Stream<Map<String, int>> watchStarsByProfile() {
    final total = attempts.starsEarned.sum();
    final query = selectOnly(attempts)
      ..addColumns([attempts.profileId, total])
      ..groupBy([attempts.profileId]);
    return query
        .map(
          (row) =>
              MapEntry(row.read(attempts.profileId)!, row.read(total) ?? 0),
        )
        .watch()
        .map(Map.fromEntries);
  }
```

ماذا يفعل؟ `watchStarsByProfile` : بث بخريطة (معرّف الملف ← مجموع النجوم) بـ `GROUP BY` .

لماذا بهذه الطريقة؟ استعلام واحد لكل الملفات لشاشة الاختيار بدل استعلام لكل بطاقة. الملف بلا محاولات لا يظهر في الخريطة (القيمة الافتراضية 0 عند القراءة).

الأسطر 110–120

```
  /// يمسح كل نشاط ملف واحد (تقدّم، محاولات، إتقان) دون حذف الملف نفسه.
  /// للاستخدام في أدوات التطوير فقط.
  Future<void> clearActivity(String profileId) async {
    await (delete(
      lessonProgress,
    )..where((r) => r.profileId.equals(profileId))).go();
    await (delete(attempts)..where((r) => r.profileId.equals(profileId))).go();
    await (delete(
      conceptMastery,
    )..where((r) => r.profileId.equals(profileId))).go();
  }
```

ماذا يفعل؟ `clearActivity`: يحذف تقدّم ومحاولات وإتقان ملف دون حذف الملف نفسه.

لماذا بهذه الطريقة؟ للاستخدام في أدوات التطوير فقط (زر «مسح نشاط هذا الملف»).

الأسطر 122–135

```
  // طابور المزامنة
  Future<void> enqueueSync({
    required String entity,
    required String payloadJson,
    required DateTime now,
  }) async {
    await into(syncQueue).insert(
      SyncQueueCompanion.insert(
        entity: entity,
        payloadJson: payloadJson,
        createdAt: now,
      ),
    );
  }
```

ماذا يفعل؟ `enqueueSync`: إضافة عنصر لطابور المزامنة.

لماذا بهذه الطريقة؟ يُملأ الآن ولا يُرسل.

الأسطر 137–143

```
  Stream<int> watchPendingSyncCount() {
    final countExp = syncQueue.id.count();
    return (selectOnly(syncQueue)..addColumns([countExp]))
        .map((row) => row.read(countExp) ?? 0)
        .watchSingle();
  }
}
```

ماذا يفعل؟ `watchPendingSyncCount`: بث بعدد العناصر المنتظرة، ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ سيظهر في شاشة «رمز الطالب» («N نتائج بانتظار الاتصال»).

lib/core/db/daos/settings_dao.dart — إعدادات الجهاز

`lib/core/db/daos/settings_dao.dart`

الأسطر 1–6

```dart
import 'package:drift/drift.dart';

import '../app_database.dart';
import '../tables/app_settings.dart';

part 'settings_dao.g.dart';
```

ماذا يفعل؟ استيرادات وجزء الكود المولَّد.

لماذا بهذه الطريقة؟ نفس النمط.

الأسطر 8–11

```dart
@DriftAccessor(tables: [AppSettings])
class SettingsDao extends DatabaseAccessor<AppDatabase>
    with _$SettingsDaoMixin {
  SettingsDao(super.db);
```

ماذا يفعل؟ `@DriftAccessor` وصنف `SettingsDao`.

لماذا بهذه الطريقة؟ يخدم جدول `app_settings`.

السطر 13

```dart
  static const activeProfileKey = 'active_profile_id';
```

ماذا يفعل؟ ثابت المفتاح `active_profile_id`.

لماذا بهذه الطريقة؟ في مكان واحد حتى لا يختلف النص بين الكتابة والقراءة.

الأسطر 15–16

```dart
  /// بادئة مفاتيح تفضيلات طالب (`pref.<id>.<name>`).
  static String profilePrefsPrefix(String profileId) => 'pref.$profileId.';
```

ماذا يفعل؟ `profilePrefsPrefix(id)`: بادئة مفاتيح تفضيلات طالب `pref.<id>.` مع تعليقها.

لماذا بهذه الطريقة؟ دالة ساكنة في مكان واحد يستعملها مستودع التفضيلات (القراءة والكتابة) ومستودع الملفات (الحذف) فلا يختلف النص بينهما أبدًا.

الأسطر 18–23

```dart
  Future<String?> read(String key) async {
    final row = await (select(
      appSettings,
    )..where((s) => s.key.equals(key))).getSingleOrNull();
    return row?.value;
  }
```

ماذا يفعل؟ `read`: قيمة مفتاح أو `null`.

لماذا بهذه الطريقة؟ `row?.value` آمن إن لم يوجد الصف.

الأسطر 25–29

```
25    Future<void> write(String key, String value) async {
26      await into(appSettings).insertOnConflictUpdate(
27        AppSettingsCompanion.insert(key: key, value: value),
28      );
29    }
```

ماذا يفعل؟ `write` : كتابة أو تحديث.

لماذا بهذه الطريقة؟ `insertOnConflictUpdate` على المفتاح الأساسي.

الأسطر 31–36

```
31    /// كل الإعدادات التي يبدأ مفتاحها بـ [prefix] (مفتاح ← قيمة)، وتتحدّث تلقائيًا.
32    Stream<Map<String, String>> watchByPrefix(String prefix) {
33      return (select(appSettings)..where((s) => s.key.like('$prefix%')))
34          .watch()
35          .map((rows) => {for (final r in rows) r.key: r.value});
36    }
```

ماذا يفعل؟ `watchByPrefix` : بث بكل الإعدادات التي يبدأ مفتاحها بنص معيّن (مفتاح ← قيمة).

لماذا بهذه الطريقة؟ `like` على بادئة المفتاح تجلب كل تفضيلات طالب بمفتاح واحد مشترك البداية `pref.<id>.` ، و `watch()` يعيد الإرسال عند أي تغيير فتتحدّث الشاشة وحجم الخط فورًا. معرّف الطالب UUID (حروف وأرقام وشرطات) فلا رموز خاصة في `LIKE` ، والنقطة قبله وبعده تمنع تداخل معرّفين أحدهما بادئة للآخر.

الأسطر 38–41

```
38    /// يحذف كل الإعدادات التي يبدأ مفتاحها بـ [prefix].
39    Future<void> removeByPrefix(String prefix) async {
40      await (delete(appSettings)..where((s) => s.key.like('$prefix%'))).go();
41    }
```

ماذا يفعل؟ `removeByPrefix` : يحذف كل الإعدادات التي يبدأ مفتاحها بنص معيّن.

لماذا بهذه الطريقة؟ يمسح تفضيلات طالب كاملة بأمر واحد عند حذف ملفه فلا تبقى بيانات يتيمة. نفس شكل `LIKE` في `watchByPrefix` .

الأسطر 43–46

```
43    Future<void> remove(String key) async {
44      await (delete(appSettings)..where((s) => s.key.equals(key))).go();
45    }
46  }
```

ماذا يفعل؟ `remove` : حذف مفتاح ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ لإلغاء إعداد مثل الملف النشط.